

Módulo 7 – Capítulo 03 – Sección 01

Anatomía de una herramienta: nombre, descripción, esquema de parámetros y respuesta

Si el razonamiento es la mente del agente, las herramientas son sus manos. El capítulo anterior estableció la calidad del razonamiento como el determinante principal de cuándo y por qué el agente decide invocar una herramienta específica. Este capítulo examina la otra mitad de esa ecuación: la calidad del diseño de las herramientas determina si esa decisión puede ejecutarse correctamente. Un agente con razonamiento excelente pero herramientas mal diseñadas produce resultados mediocres porque sus decisiones no se materializan en acciones precisas. La anatomía de una herramienta bien diseñada es el primer principio de la ingeniería de herramientas agénticas.

Una herramienta en el contexto de sistemas agénticos es una función con una interfaz definida formalmente que el LLM puede invocar mediante function calling. Esta interfaz tiene cuatro componentes: un **nombre** único que actúa como identificador semántico para el modelo, una **descripción** en lenguaje natural que el modelo usa para decidir cuándo y por qué usarla, un **esquema JSON Schema** que especifica los parámetros de entrada con sus tipos y restricciones, y una **respuesta estructurada** que el agente incorpora como observación en el contexto.

El **nombre** de la herramienta es el primer punto de contacto entre el razonamiento del agente y la implementación. Debe ser un verbo-objeto descriptivo sin espacios que comunique la acción de forma inequívoca: `search_web`, `execute_python`, `read_file`, `query_database`. Nombres genéricos como `tool1` o `helper` son invisibles para el razonamiento del LLM: el modelo no puede decidir inteligentemente entre `tool1` y `tool2` porque los nombres no comunican nada. Nombres redundantes como `search_web_search_internet_lookup` añaden ambigüedad sin claridad. La regla práctica es que el nombre solo debe leer y ya el desarrollador —y el LLM— deben poder anticipar qué hace la función.

La **descripción** es el componente más crítico y el más frecuentemente subestimado. Es el código que el LLM ejecuta internamente para decidir si esta herramienta es la apropiada para

la situación actual. Una descripción bien escrita tiene cuatro partes: qué hace la herramienta en términos concretos (no vagos), cuándo es apropiado usarla, cuándo NO usarla — especialmente cuando existe una alternativa más adecuada—, y qué limitaciones tiene (número máximo de resultados, formatos soportados, latencia esperada). La especificidad de la descripción es la diferencia entre un agente que selecciona herramientas con precisión quirúrgica y uno que invoca la herramienta incorrecta el 30% de las veces.

El **esquema JSON Schema** de parámetros especifica formalmente las entradas de la herramienta: tipo de dato (`string`, `number`, `boolean`, `array`, `object`), descripción de cada campo, valores permitidos mediante `enum`, y cuáles son requeridos. El campo `description` de cada parámetro es tan importante como el schema en sí: `"date": {"type": "string"}` no le dice al LLM qué formato usar; `"date": {"type": "string", "description": "Fecha en formato ISO 8601, ejemplo: '2024-03-15'. No usar formato MM/DD/YYYY."}` elimina la ambigüedad y reduce errores de formato que producen fallos de la herramienta.

La **respuesta** de la herramienta debe ser serializable a string para incorporarse al contexto del LLM. La regla de diseño más importante aquí es la concisión informativa: la respuesta debe contener exactamente la información que el agente necesita para razonar sobre el siguiente paso, sin verbosidad que consuma tokens sin valor. Una búsqueda web que devuelve el HTML completo de cinco páginas es una respuesta que perjudica al agente; una búsqueda que devuelve el título, URL y un snippet de 200 caracteres de cada resultado es una respuesta que facilita el razonamiento.

Componentes principales

- **Nombre:** identificador semántico verbo-objeto sin espacios (`search_web`, `execute_python`, `read_file`); comunica directamente al LLM la naturaleza de la acción disponible
- **Descripción:** texto en lenguaje natural de 50-200 palabras que especifica qué hace la herramienta, cuándo usarla, cuándo NO usarla, y sus limitaciones; es el mecanismo principal de control de selección de herramientas
- **JSON Schema de parámetros:** especificación formal de inputs con `type`, `description`, `enum` y `required`; el campo `description` de cada parámetro es tan crítico como el tipo para guiar la generación correcta de argumentos por el LLM
- **Tipo de retorno y formato:** respuesta serializable a string con la información necesaria para el siguiente paso de razonamiento; concisa y sin verbosidad que consuma tokens sin valor

- **Metadatos de herramienta:** información adicional sobre reversibilidad (read-only, reversible, irreversible), latencia esperada y tasa de error; útil para sistemas de routing de herramientas y para la política de confirmación humana

***Nota del Arquitecto:** El formato de la respuesta de herramienta que el agente recibe es tan importante como el nombre y la descripción. He visto múltiples agentes que funcionaban perfectamente en desarrollo y fallaban en producción porque la API externa cambió el formato de su respuesta JSON. La herramienta debe incluir validación de formato de respuesta y presentar al agente un formato normalizado independientemente de qué devuelva la API subyacente. Nunca exponer el formato raw de una API externa directamente al LLM.*

Principio rector

Una herramienta mal descrita es más peligrosa que una herramienta con bugs: los bugs en la implementación producen errores detectables y ruidosos, mientras que una descripción ambigua produce invocaciones incorrectas silenciosas que generan resultados plausibles pero equivocados. El diseño de la descripción merece al menos tanto tiempo como el diseño de la implementación.

La sección siguiente examina las implementaciones concretas de function calling en las tres principales plataformas de LLMs en producción, con las diferencias específicas que el desarrollador debe conocer para construir herramientas portables entre proveedores.

Módulo 7 – Capítulo 03 – Sección 02

Function calling en APIs: OpenAI, Anthropic (tool_use) y Google

La capacidad de un agente de invocar herramientas depende de cómo el proveedor del LLM expone function calling en su API. Las tres plataformas principales —OpenAI, Anthropic y Google— implementan esta funcionalidad con la misma intención pero con diferencias concretas en nomenclatura, estructura de mensajes, y comportamiento de control de flujo que pueden producir bugs sutiles cuando se construye un agente que debe ser portable o cuando se cambia de proveedor. Conocer estas diferencias no es un detalle técnico menor: es el conocimiento que permite diagnosticar por qué un agente que funciona con GPT-4o no funciona directamente con Claude sin modificaciones, y cómo diseñar la capa de abstracción que hace el código verdaderamente portable.

OpenAI (GPT-4o, GPT-4 Turbo, GPT-3.5 Turbo con soporte limitado) implementa function calling mediante el campo `tools` en la request, que contiene un array de objetos con estructura `{"type": "function", "function": {"name": "...", "description": "...", "parameters": {...JSON Schema...}}}`. Cuando el modelo decide invocar una herramienta, devuelve en el mensaje del asistente un array `tool_calls` donde cada entrada contiene un `id` único, el `function.name`, y los `function.arguments` como string JSON. El desarrollador ejecuta la función con esos argumentos y devuelve el resultado como un mensaje adicional con `role: "tool"`, el `tool_call_id` correspondiente, y el `content` con el resultado serializado. El ciclo completa cuando el modelo decide que ya tiene suficiente información para responder sin invocar más herramientas.

Anthropic (Claude 3.5 Sonnet, Claude 3 Opus, Claude 3 Haiku) usa el mismo campo `tools` en la request, pero la estructura y el flujo de mensajes son diferentes. El modelo devuelve bloques `content` de tipo `tool_use` con `id`, `name`, e `input` como objeto JSON (no como string). Los resultados se devuelven no como mensajes separados sino como bloques `tool_result` dentro del siguiente mensaje de usuario, lo que significa que el formato de la conversación alterna entre mensajes de usuario y mensajes de asistente de forma estricta. Esta diferencia de formato —tool results en el mensaje de usuario en lugar de mensajes

independientes— es la fuente más común de incompatibilidad cuando se portan agentes de OpenAI a Anthropic.

Google (Gemini 1.5 Pro, Gemini 1.5 Flash) usa `function_declarations` dentro del campo `tools`, y el modelo devuelve `Part` de tipo `functionCall` con nombre y argumentos como `Struct` protobuf. Las respuestas se envían como `Part` de tipo `functionResponse` en el siguiente turno. La principal diferencia operativa con OpenAI y Anthropic es el uso de types de protobuf en lugar de JSON nativo, lo que requiere adaptadores específicos cuando se trabaja con la librería de Python.

Las **llamadas paralelas a herramientas** (parallel tool use) son especialmente importantes para la eficiencia de los agentes. GPT-4o y Claude 3.5 Sonnet pueden invocar múltiples herramientas en una sola respuesta: en lugar de buscar en la web, esperar el resultado, y luego buscar en la base de datos, el agente puede iniciar ambas búsquedas simultáneamente. Esta capacidad puede reducir el número de iteraciones del ciclo en hasta un 50% para tareas donde múltiples acciones son independientes entre sí. El desarrollador debe estar preparado para manejar arrays de `tool_calls` en OpenAI y arrays de bloques `tool_use` en Anthropic, ejecutando cada invocación en paralelo con `asyncio.gather()` y retornando todos los resultados en el siguiente mensaje.

El parámetro **tool_choice** permite controlar el comportamiento de selección de herramientas: `tool_choice: "auto"` (el modelo decide si usar herramientas o responder directamente), `tool_choice: "none"` (el modelo debe responder sin herramientas), y `tool_choice: {"type": "tool", "name": "..."}` en Anthropic (forzar el uso de una herramienta específica). Esta última opción es útil para garantizar outputs estructurados en formato específico: si se necesita que el agente devuelva siempre un JSON validado, se puede definir una "herramienta de output" ficticia con el schema Pydantic del resultado esperado y forzar al modelo a usarla.

Aspectos técnicos

- **OpenAI tool_calls:** el modelo devuelve un array `tool_calls` con id, nombre y argumentos como string JSON; los resultados se devuelven como mensajes con `role: "tool"` referenciando el `tool_call_id`
- **Anthropic tool_use:** el modelo devuelve bloques `content` de tipo `tool_use` con id, nombre e `input` como objeto JSON; los resultados se devuelven como bloques `tool_result` dentro del siguiente mensaje de `role: "user"`

- **Google functionCall:** el modelo devuelve `Part` de tipo `functionCall` con nombre y argumentos como `Struct`; la respuesta se envía como `Part` de tipo `functionResponse` en el siguiente turno de usuario
- **tool_choice / forced calling:** las tres APIs permiten forzar el uso de una herramienta específica o prohibir el uso de herramientas; útil para garantizar outputs estructurados o para implementar herramientas de output como alternativa a JSON mode
- **Parallel tool use:** GPT-4o y Claude 3.5 Sonnet pueden invocar múltiples herramientas en una sola respuesta; ejecutar las invocaciones paralelas con `asyncio.gather()` reduce la latencia total en tareas con acciones independientes

***Nota del Arquitecto:** Cuando el proyecto requiere portabilidad entre proveedores, la solución correcta es una capa de abstracción delgada, no confiar en el que el framework lo resuelva automáticamente. LangChain y LangGraph abstraen estas diferencias razonablemente bien, pero a veces a costa de exponer menos control sobre los detalles de implementación específicos de cada proveedor. Para proyectos donde el costo de API es una variable crítica, trabajar directamente con las APIs nativas y construir una capa de abstracción propia es frecuentemente más eficiente.*

Para recordar

La portabilidad entre proveedores de function calling no es automática: las diferencias en estructura de respuesta, manejo de herramientas paralelas y control de tool_choice requieren una capa de abstracción explícita o el uso de un framework que la proporcione y que el equipo entienda suficientemente bien para depurar cuando hay comportamientos inesperados.

La sección siguiente examina por qué el diseño de la descripción de cada herramienta es tan importante como su implementación: una herramienta con implementación perfecta pero descripción ambigua producirá errores de selección que ninguna cantidad de debugging en la implementación podrá resolver.

Módulo 7 – Capítulo 03 – Sección 03

Diseño de herramientas: por qué la descripción importa tanto como la implementación

El diseño de herramientas para agentes es una forma de programación en dos lenguajes simultáneamente: el lenguaje de implementación (Python, TypeScript) para el comportamiento de la función, y el lenguaje natural para la descripción que guía al LLM en su uso. El error más frecuente en la ingeniería de agentes es invertir el 90% del tiempo en la implementación y el 10% en la descripción, cuando en producción la mayoría de los fallos de herramientas provienen no de bugs en el código sino de descripciones que el LLM interpreta de forma diferente a la intención del desarrollador. Una herramienta con implementación impecable y descripción ambigua producirá invocaciones incorrectas que el stack trace no puede ayudar a diagnosticar, porque el código funcionó perfectamente —simplemente fue invocado con los argumentos equivocados para la situación equivocada.

La descripción de una herramienta bien diseñada responde a cuatro preguntas: ¿qué hace esta herramienta en términos de efecto concreto en el mundo? ¿cuándo es apropiado usarla comparada con las alternativas disponibles? ¿qué restricciones de formato o de contenido tienen sus parámetros? ¿qué no puede hacer, cuáles son sus limitaciones conocidas? Considerar un ejemplo concreto: una herramienta de búsqueda web podría describirse como "Busca información en internet" —vaga, sin especificidad sobre la fuente, el tipo de datos o las limitaciones—, o como "Busca páginas web públicas en tiempo real usando el motor Tavily. Usa esta herramienta cuando necesites información actualizada posterior a tu fecha de entrenamiento, cuando el usuario pregunte por hechos actuales (precios, noticias, datos de empresas), o cuando necesites verificar información antes de responder. No uses esta herramienta si ya tienes la respuesta en el contexto de la conversación. Devuelve los 5 mejores resultados con título, URL y snippet de 300 caracteres. La latencia esperada es de 2-5 segundos." La segunda descripción tarda 20 minutos más en escribirse y produce agentes con tasas de selección correcta de herramientas significativamente más altas.

La **documentación de no-uso** —especificar explícitamente cuándo NO usar una herramienta— es la parte de la descripción que más frecuentemente se omite y que más

impacto tiene en reducir invocaciones innecesarias. Un agente sin esta información tiende a sobreutilizar herramientas: si tiene acceso a una herramienta de búsqueda web, puede invocarla para preguntas cuya respuesta ya está disponible en el system prompt o en el historial de la conversación, aumentando la latencia y el costo sin beneficio. La instrucción "No uses esta herramienta si ya tienes la información en el contexto" puede reducir las invocaciones innecesarias en un 20-40% según el tipo de tarea.

Las **descripciones de parámetros** merecen el mismo nivel de atención que la descripción general de la herramienta. Cada parámetro debe incluir: el tipo de valor esperado con ejemplos concretos, restricciones de formato específicas (fechas en ISO 8601, no en formato libre), el rango de valores válidos si aplica, y aclaraciones sobre casos borde comunes. Un parámetro como `language: str` sin descripción adicional genera inputs inconsistentes: el LLM puede pasar "Spanish", "Español", "es", o "ES" sin saber cuál es el formato correcto.

```
language: str, descripción: "Código de idioma ISO 639-1 de dos caracteres.  
Ejemplos: 'en' para inglés, 'es' para español, 'pt' para portugués."
```

 elimina la ambigüedad.

El **namespaceing de herramientas** se vuelve crítico cuando el agente tiene acceso a 10 o más herramientas. Sin agrupación semántica en los nombres, el espacio de herramientas se convierte en una lista plana donde el LLM tiene dificultades para distinguir entre opciones similares. Usar prefijos de dominio — `db_query`, `db_insert`, `db_update` para operaciones de base de datos; `file_read`, `file_write`, `file_list` para operaciones de filesystem; `web_search`, `web_scrape` para operaciones web— reduce la ambigüedad de selección porque el agente puede razonar a nivel de dominio antes de razonar a nivel de operación específica.

La **evaluación de selección de herramientas** mediante un banco de pruebas es la única forma objetiva de verificar que las descripciones funcionan como se espera. Construir un dataset de 20-50 escenarios con el input del usuario y la herramienta esperada, ejecutarlo contra el agente con herramientas mockeadas, y medir la precisión de selección es un proceso que tarda menos de un día en implementarse y puede detectar problemas de descripción antes de que lleguen a producción. Los cambios de versión del modelo base —especialmente upgrades— deben tratarse como cambios que pueden alterar el comportamiento de selección y deben re-validarse contra este banco de pruebas.

Puntos críticos

- **Descripción como contrato de API:** tratar la descripción de cada herramienta con la misma precisión que un docstring de una función pública en una librería; especificidad, completitud y ausencia de ambigüedad son requisitos, no aspiraciones
- **Documentación de no-uso:** especificar explícitamente cuándo NO usar la herramienta reduce invocaciones innecesarias en 20-40%; tan importante como especificar cuándo sí usarla
- **Descripciones de parámetros con ejemplos:** cada parámetro debe incluir tipo, descripción, formato esperado con ejemplos concretos, y restricciones de rango; `date:` `str` nunca es suficiente si el formato importa
- **Namespacing semántico:** usar prefijos de dominio en nombres de herramientas (`db_` , `file_` , `web_`) reduce la ambigüedad de selección en agentes con más de 10 herramientas disponibles
- **Banco de pruebas de selección:** dataset de 20-50 escenarios con herramienta esperada; ejecutar en CI/CD ante cualquier cambio de descripción o de modelo base; la precisión de selección es la primera métrica de calidad de las herramientas

Buena práctica

Tratar las descripciones de herramientas como contratos de API: deben ser tan precisas, completas y sin ambigüedad como los docstrings de una función pública en una librería que usan otros desarrolladores. Si un colega no puede leer la descripción y saber exactamente cuándo usar la herramienta y cuándo no, el LLM tampoco puede.

La sección siguiente examina las herramientas de sistema —shell, filesystem, navegador web e intérpretes de código— que son las más potentes del arsenal de un agente y también las que requieren las políticas de seguridad más estrictas.

Módulo 7 – Capítulo 03 – Sección 04

Herramientas de sistema: shell, filesystem, navegador web y code interpreter

Las herramientas de sistema son las más potentes del arsenal de un agente y, por la misma razón, las que requieren el diseño de seguridad más riguroso. Mientras que una herramienta de búsqueda web que devuelve resultados incorrectos produce una mala respuesta, una herramienta de shell que ejecuta el comando incorrecto puede comprometer el sistema host, y una herramienta de base de datos que ejecuta el SQL incorrecto puede borrar datos de producción de forma irreversible. El poder de las herramientas de sistema es proporcional a la responsabilidad de su diseño: cada herramienta que da acceso al sistema operativo, el filesystem, la red o la ejecución de código debe incluir controles de seguridad que operen de forma independiente del razonamiento del LLM.

El **intérprete de código** (code interpreter) es la herramienta de mayor versatilidad: permite al agente escribir y ejecutar código Python para manipular datos, realizar cálculos, generar visualizaciones, o interactuar con librerías que no tienen una herramienta específica disponible. La implementación correcta en producción requiere un **sandbox aislado**: el código generado por el LLM no debe ejecutarse en el mismo proceso que el agente ni en el sistema host. Las plataformas especializadas en sandboxing de código para agentes incluyen **E2B** (e2b.dev), que provee microVMs Linux efímeras con API de Python (`sandbox = e2b.Sandbox(); sandbox.process.start_and_wait("python script.py")`), **Modal** (modal.com), que ofrece contenedores serverless con runtimes preconfigurados, y **Daytona**, orientado a entornos de desarrollo completos. Para implementaciones self-hosted, Docker con restricciones de red (`--network none`) y de filesystem (montaje read-only excepto el directorio de trabajo) es la opción más común. El timeout de ejecución debe ser explícito y proporcionado a la complejidad esperada de los scripts: 30 segundos para scripts de manipulación de datos, 5 minutos para pipelines más complejos.

La **herramienta de filesystem** da al agente acceso a leer y escribir archivos, lo que puede ser necesario para procesar documentos subidos por el usuario, guardar resultados intermedios, o interactuar con datos locales. El control de acceso es crítico: la herramienta

debe restringirse a un directorio de trabajo definido y rechazar activamente rutas absolutas, path traversal (`../../../../etc/passwd`), y symlinks que apunten fuera del directorio autorizado. Esta validación debe ocurrir en el código Python de la herramienta antes de cualquier operación de filesystem, no en el system prompt del agente. Un agente que confía en que el LLM "nunca pedirá acceso a rutas no autorizadas" está un prompt injection de distancia de un incidente de seguridad.

La **herramienta de shell** es la de mayor superficie de ataque de todas: ejecutar comandos arbitrarios del sistema operativo puede hacer prácticamente cualquier cosa que el usuario del proceso tenga permisos para hacer. Su uso en agentes de producción debería ser la excepción, no la regla, y cuando se justifica, debe implementarse con: una lista blanca de comandos permitidos (verificada antes de la ejecución), timeout estricto (máximo 30 segundos), ejecución como usuario sin privilegios (nunca como root), y logging inmutable de cada comando ejecutado con timestamp, usuario y output. El monto de comandos que pueden ejecutarse con seguridad absoluta en producción es pequeño; para la mayoría de las operaciones de sistema, es mejor diseñar herramientas Python específicas que encapsulen las operaciones necesarias en lugar de exponer un shell genérico.

Las **herramientas de navegador web** permiten al agente interactuar con páginas dinámicas que los scrapers HTTP simples no pueden acceder: formularios, páginas renderizadas por JavaScript, sistemas que requieren autenticación, o interfaces web que no tienen API pública. **Playwright** headless es la herramienta estándar de la industria para este propósito: permite navegar, hacer clic, rellenar formularios y extraer contenido de páginas web con una API Python de alto nivel. Para agentes que solo necesitan acceso a contenido indexable, las **APIs de búsqueda web** como Tavily Search, SerpAPI o la API de Bing son preferibles: devuelven resultados estructurados sin el overhead de renderizar páginas, con latencia de 1-3 segundos en lugar de los 5-15 segundos de una navegación con Playwright. La regla práctica es usar una API de búsqueda web cuando el agente necesita encontrar información, y usar Playwright solo cuando necesita interactuar con la interfaz de una página específica.

Componentes principales

- **Python REPL / Code interpreter:** ejecución de código generado por el LLM en un sandbox aislado; E2B Sandbox, Modal y Daytona ofrecen entornos cloud con aislamiento de contenedor, timeout configurable y API de alto nivel; para self-hosted, Docker con `--network none` y usuario sin privilegios

- **Herramienta de filesystem:** operaciones de lectura/escritura de archivos; restringida a un directorio de trabajo definido; validar y rechazar path traversal y rutas absolutas en el código Python de la herramienta, no en el prompt del agente
- **Shell tool:** ejecución de comandos del sistema; la herramienta de mayor superficie de ataque; lista blanca de comandos permitidos, timeout máximo de 30 segundos, nunca ejecutar como root; preferir herramientas Python específicas sobre un shell genérico
- **Navegador web (Playwright):** navegación de páginas dinámicas con JavaScript; bloquear recursos no necesarios (imágenes, fuentes, media) para reducir latencia; timeout de 10-30 segundos por página; preferir APIs de búsqueda cuando solo se necesita contenido indexado
- **APIs de búsqueda web (Tavily, SerpAPI, Bing):** resultados estructurados sin overhead de renderizado; latencia de 1-3 segundos; opción preferida cuando el agente necesita información actualizada sin interacción con la interfaz de una página específica

Nota del Arquitecto: La regla que más me ha servido en el diseño de herramientas de sistema es esta: cualquier herramienta que, si es comprometida por prompt injection, puede causar daño irreversible necesita una capa adicional de protección independiente del LLM. Esa protección puede ser confirmación humana explícita, verificación programática de los argumentos antes de ejecutar, o restricción del scope de la herramienta a un nivel donde el daño máximo posible sea aceptable. Diseñar esta restricción en la implementación, no en el prompt.

Buena práctica

Aplicar el principio de mínimo privilegio a cada herramienta de sistema: si el agente solo necesita leer archivos CSV de un directorio específico, la herramienta debe rechazar cualquier operación fuera de ese scope antes de enviarse al ejecutor, no después. La validación de scope debe ser parte del código de la herramienta, no una restricción expresada solo en el system prompt del agente.

La sección siguiente aborda el estándar emergente que está formalizando la interfaz entre LLMs y herramientas externas: el Model Context Protocol, que permite construir herramientas reutilizables entre agentes y publicarlas como servicios independientes.

Módulo 7 – Capítulo 03 – Sección 05

Estándares de herramientas: MCP y la estandarización del tool use

Las secciones anteriores de este capítulo establecieron los principios del diseño de herramientas: nombre inequívoco, descripción precisa, schema JSON Schema bien documentado, y respuesta concisa. Estos principios son necesarios pero sufren de un problema de fragmentación: cada agente, cada framework y cada equipo implementa herramientas con su propia convención de nombres, estructura de parámetros y mecanismo de transporte. Una herramienta de búsqueda web desarrollada para un agente LangGraph no es directamente reutilizable por un agente AutoGen sin adaptación manual. El Model Context Protocol (MCP) es la respuesta de la industria a este problema de fragmentación: un estándar abierto que define cómo los LLMs se comunican con herramientas externas de forma portátil, reutilizable y componible.

MCP (Model Context Protocol), publicado por Anthropic en noviembre de 2024 como especificación abierta, define un protocolo de comunicación entre un cliente LLM (el agente que invoca herramientas) y un servidor MCP (el servicio que implementa las herramientas). La arquitectura es cliente-servidor: el servidor MCP expone un conjunto de herramientas con sus schemas y descripciones a través de una interfaz estándar; el cliente LLM descubre las herramientas disponibles, las incluye en el contexto del modelo, y envía los argumentos de invocación al servidor cuando el LLM decide usar una herramienta. El transporte puede ser local (stdio, para servidores MCP que corren en el mismo proceso) o remoto (HTTP/SSE, para servidores MCP expuestos como servicios independientes).

La diferencia fundamental entre MCP y el function calling directo de OpenAI o Anthropic es el nivel de abstracción. Con function calling directo, las herramientas son funciones Python definidas en el mismo proceso del agente: el desarrollador escribe la función, genera el schema JSON, y pasa ambos al modelo en cada request. Con MCP, las herramientas son servicios independientes que el agente descubre dinámicamente: en lugar de hardcodear las herramientas en el código del agente, el agente se conecta a un servidor MCP y pregunta qué herramientas están disponibles. Esto tiene consecuencias prácticas importantes para la

reutilización: una herramienta de búsqueda en GitHub implementada como servidor MCP puede ser usada por cualquier agente compatible con MCP —en cualquier framework, en cualquier lenguaje— sin duplicar el código de la implementación.

Implementar un servidor MCP básico en Python requiere el paquete `mcp` del repositorio oficial de Anthropic. La estructura mínima de un servidor MCP define herramientas mediante decoradores:

```
from mcp.server import Server
from mcp.server.stdio import stdio_server
from mcp.types import Tool, TextContent

server = Server("my-tools-server")

@server.list_tools()
async def list_tools():
    return [
        Tool(
            name="search_documentation",
            description="Busca en la documentación interna del proyecto. Usa esto cuando",
            inputSchema={
                "type": "object",
                "properties": {
                    "query": {"type": "string", "description": "Términos de búsqueda en"},
                    "max_results": {"type": "integer", "description": "Número máximo de"},
                },
                "required": ["query"]
            }
        )
    ]

@server.call_tool()
async def call_tool(name: str, arguments: dict):
    if name == "search_documentation":
        results = await search_internal_docs(arguments["query"])
        return [TextContent(type="text", text=format_results(results))]
```

El servidor se conecta al cliente agente mediante `stdio_server()` para el caso local o mediante HTTP para el caso remoto. LangGraph, Claude Desktop y otros clientes compatibles con MCP pueden conectarse a este servidor y obtener automáticamente la lista de herramientas disponibles sin que el desarrollador del agente necesite conocer los detalles de implementación del servidor.

Los casos donde MCP añade valor sobre function calling directo son específicos y claros. Primero, cuando el mismo conjunto de herramientas debe ser reutilizable entre múltiples agentes o proyectos: en lugar de duplicar la implementación, todos los agentes apuntan al mismo servidor MCP. Segundo, cuando el ecosistema de herramientas es grande y dinámico: los servidores MCP pueden exponer docenas de herramientas que el agente descubre en tiempo de ejecución, en lugar de tener todas hardcodeadas en el sistema prompt. Tercero, cuando las herramientas deben ser mantenidas por equipos diferentes del equipo de agentes: el servidor MCP como servicio independiente tiene su propio ciclo de vida, tests y deployments.

La **gestión de errores en herramientas MCP** sigue los mismos principios que cualquier herramienta agéntica, pero con la particularidad de que los errores de comunicación con el servidor MCP son una categoría adicional: timeout de conexión, servidor no disponible, o respuesta mal formada. El cliente MCP debe implementar reintentos con backoff exponencial para fallos de conectividad transitorios, y comunicar al agente de forma informativa cuándo el servidor no está disponible para que pueda intentar herramientas alternativas. Los mensajes de error devueltos al LLM deben incluir el tipo de fallo (error de red, error de la herramienta, timeout) y una sugerencia de acción: "El servidor de herramientas de documentación no está disponible. Considera usar la herramienta de búsqueda web como alternativa o informar al usuario que la búsqueda en documentación interna no está disponible temporalmente."

Conceptos clave

- **MCP (Model Context Protocol):** estándar abierto publicado por Anthropic en 2024; define la comunicación entre clientes LLM y servidores de herramientas mediante un protocolo cliente-servidor portátil entre frameworks y proveedores
- **Servidor MCP:** servicio que expone herramientas con nombre, descripción y schema JSON; el agente descubre las herramientas dinámicamente en lugar de tenerlas hardcodeadas; el transporte puede ser stdio (local) o HTTP/SSE (remoto)
- **Cliente MCP:** el componente del agente que se conecta al servidor, descubre herramientas, y envía invocaciones; soportado nativamente en Claude Desktop, LangGraph y en la API de Anthropic con el parámetro `betas: ["computer-use-2024-10-22"]`
- **Reutilización entre agentes:** el principal beneficio de MCP sobre function calling directo; una implementación de herramienta en un servidor MCP es accesible por cualquier agente compatible sin duplicar código

- **Ecosistema de servidores MCP:** la comunidad ha publicado cientos de servidores MCP pre-construidos (GitHub, Notion, Slack, PostgreSQL, filesystem, web search) disponibles como paquetes npm y PyPI; reutilizar servidores pre-construidos antes de implementar desde cero
- **Retry y graceful degradation:** los fallos de comunicación con servidores MCP son una categoría adicional de error a manejar; implementar retry con backoff para errores transitorios y comunicar claramente al agente cuando un servidor no está disponible

***Nota del Arquitecto:** MCP resuelve un problema real de fragmentación del ecosistema, pero no es la solución correcta para todos los proyectos. Para agentes con 3-5 herramientas simples que son específicas del proyecto, function calling directo con funciones Python es más simple, más rápido de implementar, y más fácil de depurar. MCP justifica su overhead de infraestructura cuando las herramientas necesitan ser reutilizables entre proyectos o cuando el equipo de herramientas y el equipo de agentes son distintos y necesitan ciclos de vida independientes.*

Para recordar

MCP estandariza la interfaz entre LLMs y herramientas de la misma forma en que REST estandarizó la interfaz entre servicios web: no elimina la necesidad de diseñar buenas herramientas (el nombre, la descripción y el schema siguen siendo igualmente críticos), pero sí elimina la necesidad de reimplementar esa interfaz para cada combinación de agente y herramienta.

La sección siguiente cierra el capítulo sobre herramientas, articulando el principio arquitectónico que conecta el diseño de herramientas individuales con la confiabilidad del sistema agéntico completo.

Módulo 7 – Capítulo 03 – Sección 06

Cierre: las herramientas son la interfaz entre el razonamiento del agente y el mundo real

Las herramientas son el punto de contacto entre el razonamiento interno del agente —que ocurre en el espacio de tokens del LLM— y el mundo externo donde existen APIs, bases de datos, sistemas de archivos y usuarios. Esta posición de interfaz hace que las herramientas sean exactamente tan importantes como el razonamiento que las guía: un agente con razonamiento excelente pero herramientas mal diseñadas producirá resultados mediocres porque sus decisiones no podrán ejecutarse con precisión; herramientas bien diseñadas, por su parte, pueden compensar parcialmente debilidades en el razonamiento porque descripciones precisas guían al modelo hacia el uso correcto incluso en casos de borde que el prompt de razonamiento no anticipó.

El capítulo ha establecido que el diseño de herramientas opera simultáneamente en dos dominios: es ingeniería de software (la implementación debe ser robusta, manejar errores predeciblemente, validar inputs y producir outputs bien formateados) y es ingeniería de prompts (la descripción debe ser inequívoca, especificar cuándo usar y cuándo no usar la herramienta, y documentar los parámetros con ejemplos concretos). Los bugs en ambas dimensiones tienen consecuencias igualmente reales en producción: un bug de implementación produce errores visibles que el stack trace puede diagnosticar; una descripción deficiente produce invocaciones incorrectas silenciosas que el stack trace no puede ayudar a detectar porque el código funcionó perfectamente —simplemente fue invocado en el momento o con los argumentos equivocados.

La gestión de errores merece mención específica como la propiedad que distingue herramientas de producción de herramientas de prototipo. Una herramienta que solo funciona cuando el entorno externo coopera perfectamente —cuando la API devuelve HTTP 200, cuando el filesystem no tiene permisos denegados, cuando la búsqueda web devuelve resultados relevantes— no está lista para producción. Las herramientas de producción implementan políticas de error explícitas: retry con backoff exponencial para fallos transitorios (timeouts, HTTP 429 de rate limiting), fallback a herramientas alternativas

cuando la primaria no está disponible, mensajes de error informativos al LLM que explican qué falló y qué debería intentar en lugar, y degradación graceful cuando ninguna alternativa funciona. Estas políticas deben diseñarse antes de que ocurran los primeros fallos, no como reacción a ellos.

El estándar Model Context Protocol, cubierto en la sección anterior, añade una capa a este cuadro: el ecosistema de herramientas no necesita ser construido desde cero en cada proyecto. Cientos de servidores MCP pre-construidos —para GitHub, Notion, Slack, PostgreSQL, búsqueda web, filesystem— están disponibles como paquetes que pueden conectarse a cualquier agente compatible. Antes de implementar una herramienta desde cero, la primera pregunta debe ser si existe un servidor MCP que cumpla la función necesaria. La segunda pregunta debe ser si la herramienta puede construirse como un servidor MCP para que sea reutilizable en proyectos futuros.

La transición del capítulo de herramientas al capítulo de memoria es natural: las herramientas permiten al agente actuar en el mundo; la memoria permite al agente aprender de esas acciones y aplicar ese aprendizaje en interacciones futuras. Las herramientas sin memoria producen un agente que hace lo mismo en cada sesión sin importar lo que aprendió en las sesiones anteriores; la memoria sin herramientas produce un agente que sabe mucho pero no puede hacer nada con ese conocimiento. La sinergia entre ambos componentes es lo que produce agentes genuinamente útiles en dominios complejos.

Principio rector

Las herramientas son código; sus descripciones son prompts; y los bugs en ambas tienen consecuencias igualmente reales en el comportamiento del agente en producción. Tratar cada herramienta como un microservicio con su contrato de API, sus SLOs de latencia y sus políticas de error es la práctica correcta, no una aspiración de equipos con recursos abundantes.

"Any sufficiently advanced technology is indistinguishable from magic. But magic that fails silently is just a bug with a good story." — paráfrasis del tercer postulado de Arthur C. Clarke aplicada al contexto de tool use agéntico: la aparente magia de function calling —el LLM "sabe" qué herramienta usar y "sabe" cómo usarla— es el resultado de descripciones cuidadosamente diseñadas y schemas precisamente documentados, no de capacidades misteriosas del modelo. Cuando esa magia falla silenciosamente, la causa casi siempre está en la descripción o en el manejo de errores, no en el LLM.